1# **Lekcja 05: Kolekcje I – Listy i Łańcuchy Znaków w Data Science**

`#lekcja` `#datascience` `#python` `#kolekcje` `#listy` `#stringi`

W tej lekcji poznasz kluczowe struktury danych w Pythonie: **listy** i **łańcuchy znaków** (stringi). Nauczysz się efektywnie przetwarzać dane sekwencyjne, które są fundamentem analizy danych w Data Science. Zrozumiesz różnice między mutowalnością list a niemutowalnością stringów oraz poznasz najważniejsze metody do manipulacji tymi strukturami.

---

1. Listy w Pythonie – Podstawy i Indeksowanie

Definition

Lista to uporządkowana, mutowalna kolekcja elementów, która może przechowywać różne typy danych. W Data Science listy są używane do przechowywania serii pomiarów, wyników eksperymentów, nazw kolumn czy parametrów modeli.

Wprowadzenie

Listy są jedną z najbardziej uniwersalnych struktur danych w Pythonie. Dzięki możliwości modyfikacji (mutowalność) pozwalają na dynamiczne zarządzanie danymi podczas analizy. W praktyce Data Science listy często służą do:

- Przechowywania wyników obliczeń
- Budowania zbiorów danych przed konwersją do NumPy/Pandas
- Iteracji po nazwach plików, kolumn czy parametrów

Przykład 1: Tworzenie list i podstawowe operacje

```
# Tworzenie listy z pomiarami temperatury (°C) z różnych dni
temperatury = [22.5, 23.1, 21.8, 24.3, 22.9, 23.5, 22.0]
print(f"Pomiary temperatury: {temperatury}")
print(f"Liczba pomiarów: {len(temperatury)}")

# Tworzenie listy z nazwami kolumn datasetu
kolumny = ['temperatura', 'wilgotnosc', 'cisnienie', 'predkosc_wiatru']
print(f"Kolumny datasetu: {kolumny}")

# Lista może zawierać różne typy danych
```

```python
mixed_data = [42, 3.14, "Data Science", True, None]
print(f"Lista z różnymi typami: {mixed_data}")

# Tworzenie pustej listy i dodawanie elementów
wyniki_eksperymentu = []
wyniki_eksperymentu.append(0.85)  # accuracy pierwszego modelu
wyniki_eksperymentu.append(0.88)  # accuracy drugiego modelu
print(f"Wyniki modeli: {wyniki_eksperymentu}")
```

Wyjaśnienie: Listy tworzymy za pomocą nawiasów kwadratowych `[]`. Mogą zawierać elementy różnych typów, co jest przydatne w eksploracyjnej analizie danych. Funkcja `len()` zwraca liczbę elementów.

Przykład 2: Indeksowanie i wycinki (slicing)

```python
# Dataset z wartościami sprzedaży (w tysiącach) dla 12 miesięcy
sprzedaz = [120, 135, 142, 138, 155, 168, 175, 182, 190, 185, 195, 210]

# Indeksowanie dodatnie (od początku)
print(f"Sprzedaż w styczniu (indeks 0): {sprzedaz[0]} tys.")
print(f"Sprzedaż w grudniu (indeks 11): {sprzedaz[11]} tys.")

# Indeksowanie ujemne (od końca)
print(f"Ostatni miesiąc (indeks -1): {sprzedaz[-1]} tys.")
print(f"Przedostatni miesiąc (indeks -2): {sprzedaz[-2]} tys.")

# Wycinki (slicing) - pobieranie fragmentów listy
# Format: lista[start:stop:step]
q1_sprzedaz = sprzedaz[0:3]  # Pierwsze 3 miesiące (Q1)
print(f"Sprzedaż Q1: {q1_sprzedaz}")

q4_sprzedaz = sprzedaz[9:12]  # Ostatnie 3 miesiące (Q4)
print(f"Sprzedaż Q4: {q4_sprzedaz}")

# Co drugi miesiąc
co_drugi_miesiac = sprzedaz[::2]
print(f"Co drugi miesiąc: {co_drugi_miesiac}")

# Odwrócenie listy
sprzedaz_odwrocona = sprzedaz[::-1]
print(f"Sprzedaż od grudnia do stycznia: {sprzedaz_odwrocona}")

# Ostatnie 6 miesięcy
polroczne = sprzedaz[-6:]
print(f"Ostatnie półrocze: {polroczne}")
```

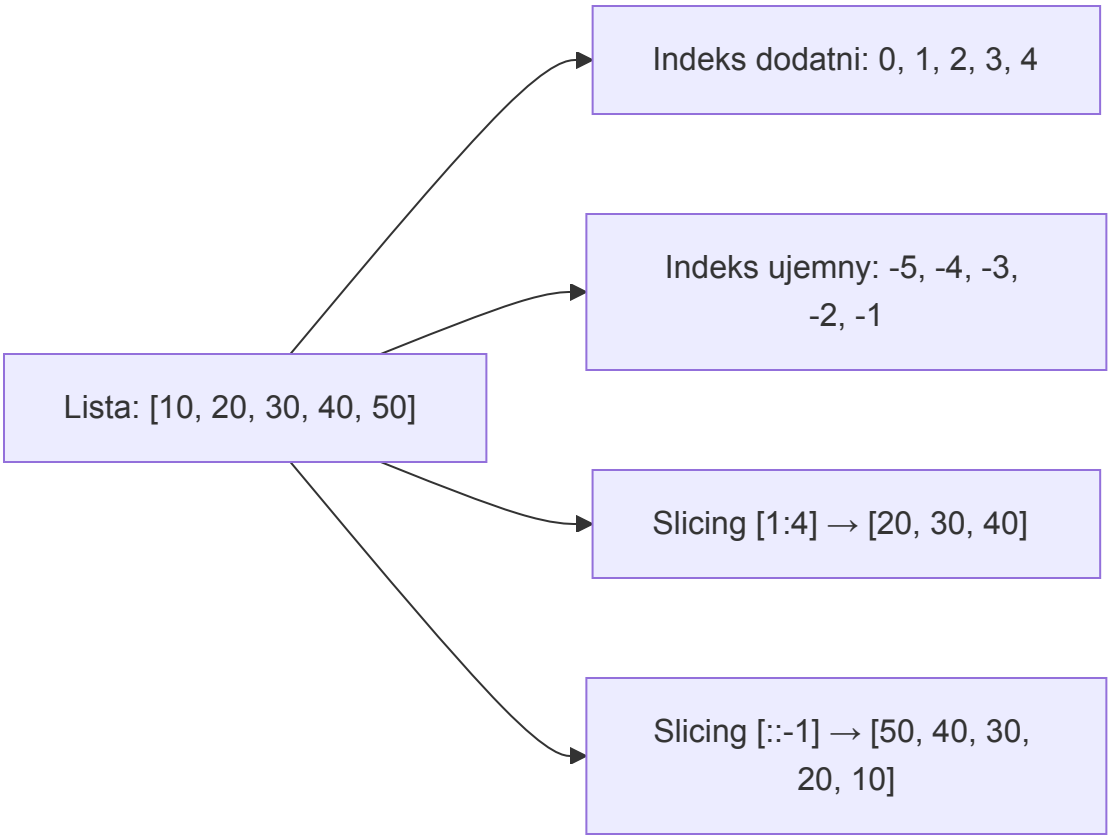
Wyjaśnienie: Indeksowanie w Pythonie zaczyna się od 0. Indeksy ujemne liczą od końca listy (-1 to ostatni element). Slicing pozwala wyciąć fragment listy w formacie `[start:stop:step]`, gdzie `stop` nie jest włączony do wyniku.

Przykład 3: Mutowalność list - modyfikacja danych in-place

```
# Dataset z wynikami accuracy modeli
accuracy_scores = [0.72, 0.68, 0.75, 0.81, 0.79]
print(f"Oryginalne wyniki: {accuracy_scores}")

# Zmiana pojedynczego elementu (poprawka błędnego pomiaru)
accuracy_scores[1] = 0.71  # Korekta drugiego wyniku
print(f"Po korekcie: {accuracy_scores}")

# Zmiana fragmentu listy
accuracy_scores[2:4] = [0.76, 0.82]  # Aktualizacja wyników 3 i 4
print(f"Po aktualizacji: {accuracy_scores}")

# Usunięcie elementu przez del
del accuracy_scores[0]  # Usunięcie pierwszego (najgorszego) wyniku
print(f"Po usunięciu pierwszego: {accuracy_scores}")

# Lista list - macierz danych (confusion matrix)
confusion_matrix = [
    [50, 5],   # Klasa 0: 50 True Negative, 5 False Positive
    [3, 42]    # Klasa 1: 3 False Negative, 42 True Positive
]
print(f"Confusion Matrix:")
print(f"  TN: {confusion_matrix[0][0]}, FP: {confusion_matrix[0][1]}")
print(f"  FN: {confusion_matrix[1][0]}, TP: {confusion_matrix[1][1]}")

# Obliczenie accuracy z confusion matrix
tn, fp = confusion_matrix[0]
fn, tp = confusion_matrix[1]
accuracy = (tp + tn) / (tp + tn + fp + fn)
print(f"Accuracy: {accuracy:.2%}")
```

Wyjaśnienie: Listy są mutowalne - możemy zmieniać ich zawartość po utworzeniu. To ważne w Data Science, gdy aktualizujemy wyniki eksperymentów czy parametry modeli. Listy mogą zawierać inne listy, tworząc struktury wielowymiarowe.

Wizualizacja: Indeksowanie i slicing



Tip

W Data Science często używamy slicingu do podziału danych na zbiory treningowe i testowe: `train = data[:800]` oraz `test = data[800:]`.

Warning

Uważaj na zakres indeksów! Python nie zgłasza błędu przy slicingu poza zakresem (zwraca pustą listę), ale zgłasza `IndexError` przy pojedynczym indeksie poza zakresem.

---

2. Metody List i Typowe Wzorce

Definition

Metody list to wbudowane funkcje, które pozwalają na efektywną manipulację danymi: dodawanie, usuwanie, sortowanie i przeszukiwanie elementów.

Wprowadzenie

Python oferuje bogaty zestaw metod do pracy z listami. W Data Science najczęściej używamy metod do agregacji danych, budowania zbiorów cech (features) oraz przygotowania danych do analizy.

Przykład 1: Dodawanie elementów - append, extend, insert

```
# Scenariusz: Zbieranie wyników accuracy z kolejnych eksperymentów

# Inicjalizacja pustej listy
experiment_results = []

# append() - dodaje jeden element na końcu
experiment_results.append(0.82)
experiment_results.append(0.85)
experiment_results.append(0.83)
print(f"Po 3 eksperymentach: {experiment_results}")

# extend() - dodaje wiele elementów z innej listy/iterowalnej
nowe_wyniki = [0.86, 0.84, 0.87]
experiment_results.extend(nowe_wyniki)
print(f"Po extend: {experiment_results}")

# Różnica między append a extend
lista_a = [1, 2, 3]
lista_b = [1, 2, 3]

lista_a.append([4, 5])  # Dodaje całą listę jako jeden element
lista_b.extend([4, 5])  # Dodaje elementy listy osobno

print(f"Po append([4,5]): {lista_a}")   # [1, 2, 3, [4, 5]]
print(f"Po extend([4,5]): {lista_b}")   # [1, 2, 3, 4, 5]

# insert() - wstawia element na konkretnej pozycji
features = ['wiek', 'dochod', 'wyksztalcenie']
features.insert(1, 'plec')  # Wstawienie 'plec' na pozycję 1
print(f"Po insert: {features}")

# Operator + do konkatenacji (tworzy NOWĄ listę)
all_features = features + ['miasto', 'zawod']
print(f"Wszystkie cechy: {all_features}")
print(f"Oryginalna lista features: {features}")  # Niezmieniona!
```

Wyjaśnienie: `append()` dodaje element na końcu (O(1) - bardzo szybko). `extend()` dodaje wiele elementów z innej sekwencji. `insert()` wstawia na konkretnej pozycji (wolniejsze dla dużych list). Operator `+` tworzy nową listę, więc nie modyfikuje oryginału.

Przykład 2: Usuwanie elementów - remove, pop, clear

```
# Dataset z wartościami odstającymi (outliers), które należy usunąć
measurements = [23.5, 24.1, 23.8, 150.0, 24.3, 23.9, -10.5, 24.0]
print(f"Oryginalne pomiary: {measurements}")

# remove() - usuwa pierwsze wystąpienie wartości
measurements.remove(150.0)  # Usunięcie outliera
print(f"Po remove(150.0): {measurements}")

# pop() - usuwa element na danej pozycji i ZWRACA go
outlier = measurements.pop(5)  # Indeks 5: -10.5
print(f"Usunięty outlier: {outlier}")
print(f"Po pop(5): {measurements}")

# pop() bez argumentu usuwa ostatni element
last = measurements.pop()
print(f"Usunięty ostatni: {last}")
print(f"Aktualna lista: {measurements}")

# clear() - usuwa wszystkie elementy
temp_list = [1, 2, 3, 4, 5]
temp_list.clear()
print(f"Po clear(): {temp_list}")  # []

# Usuwanie przez slicing i przypisanie pustej listy
data = [10, 20, 30, 40, 50, 60, 70]
data[2:5] = []  # Usunięcie elementów od indeksu 2 do 4
print(f"Po usunięciu przez slicing: {data}")
```

Wyjaśnienie: `remove(value)` usuwa pierwsze wystąpienie wartości (zgłasza `ValueError` jeśli nie istnieje). `pop(index)` usuwa i zwraca element (użyteczne gdy potrzebujemy wartości). `clear()` czyści całą listę.

Przykład 3: Sortowanie i odwracanie - sort, sorted, reverse

```
import random

# Wyniki accuracy z różnych modeli (nieposortowane)
model_scores = [0.78, 0.92, 0.85, 0.88, 0.81, 0.95, 0.79]
print(f"Oryginalne wyniki: {model_scores}")

# sort() - sortuje listę IN-PLACE (modyfikuje oryginalną listę)
model_scores.sort()
print(f"Po sort() rosnąco: {model_scores}")
```

```
model_scores.sort(reverse=True)
print(f"Po sort(reverse=True) malejąco: {model_scores}")

# sorted() - zwraca NOWĄ posortowaną listę (nie modyfikuje oryginału)
scores_original = [0.78, 0.92, 0.85, 0.88]
scores_sorted = sorted(scores_original)
print(f"Oryginalna: {scores_original}")
print(f"Posortowana (sorted): {scores_sorted}")

# Sortowanie z kluczem - przykład z tuple (model_name, accuracy)
models = [
    ('LogisticRegression', 0.78),
    ('RandomForest', 0.92),
    ('SVM', 0.85),
    ('XGBoost', 0.95)
]

# Sortowanie po accuracy (drugi element tuple)
models.sort(key=lambda x: x[1], reverse=True)
print(f"\nModele posortowane po accuracy:")
for model_name, acc in models:
    print(f"  {model_name}: {acc:.2%}")

# reverse() - odwraca kolejność IN-PLACE
numbers = [1, 2, 3, 4, 5]
numbers.reverse()
print(f"Po reverse(): {numbers}")

# Generowanie losowych danych i sortowanie
random.seed(42)  # Dla reprodukowalności
random_data = [random.randint(50, 100) for _ in range(10)]
print(f"\nLosowe dane: {random_data}")
print(f"Min: {min(random_data)}, Max: {max(random_data)}, Avg:
{sum(random_data)/len(random_data):.1f}")
```

Wyjaśnienie: `sort()` modyfikuje listę (szybsze, oszczędniejsze pamięciowo), `sorted()` tworzy nową (bezpieczniejsze dla oryginalnych danych). Parametr `key` pozwala określić funkcję do sortowania złożonych struktur.

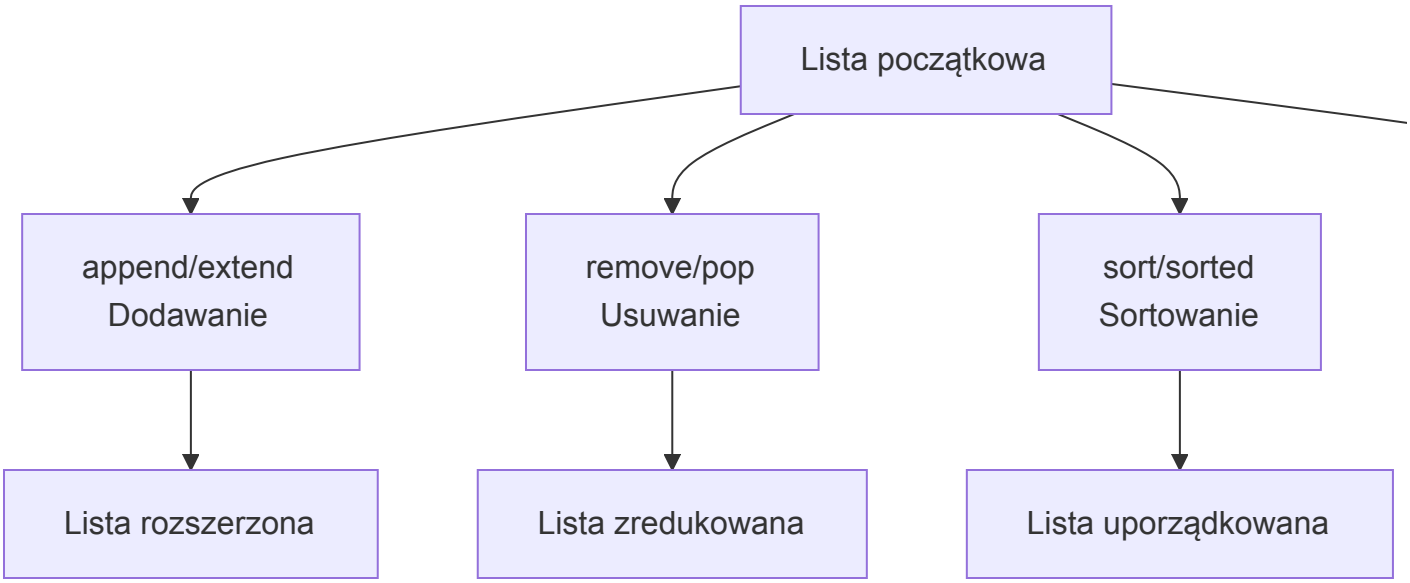
Przykład 4: Przeszukiwanie i liczenie - index, count, in

```
# Dataset z klasami przewidywanymi przez model
predictions = ['cat', 'dog', 'cat', 'bird', 'dog', 'cat', 'dog', 'cat',
'bird']
```

```python
print(f"Predykcje: {predictions}")

# count() - zlicza wystąpienia wartości
cat_count = predictions.count('cat')
dog_count = predictions.count('dog')
bird_count = predictions.count('bird')
print(f"Rozkład klas: cat={cat_count}, dog={dog_count}, bird={bird_count}")

# index() - zwraca indeks pierwszego wystąpienia
first_bird_index = predictions.index('bird')
print(f"Pierwszy 'bird' na pozycji: {first_bird_index}")

# index() z zakresem wyszukiwania
second_bird_index = predictions.index('bird', first_bird_index + 1)
print(f"Drugi 'bird' na pozycji: {second_bird_index}")

# Operator 'in' - sprawdzenie czy element istnieje
if 'cat' in predictions:
    print("Model przewidział przynajmniej jednego kota")

if 'elephant' not in predictions:
    print("Model nie przewidział słonia")

# Praktyczne zastosowanie - analiza rozkładu klas
unique_classes = []
for pred in predictions:
    if pred not in unique_classes:
        unique_classes.append(pred)

print(f"\nUnique classes: {unique_classes}")

# Liczenie wystąpień wszystkich klas
class_distribution = {}
for cls in unique_classes:
    class_distribution[cls] = predictions.count(cls)

print(f"Rozkład klas: {class_distribution}")
```

Wyjaśnienie: `count()` jest przydatne do analizy rozkładu wartości. `index()` znajduje pozycję elementu (zgłasza `ValueError` jeśli nie istnieje). Operator `in` to najszybszy sposób sprawdzenia obecności elementu.

Wizualizacja: Operacje na listach



✓ Success

Best practice w Data Science: Używaj `sorted()` zamiast `sort()` gdy potrzebujesz zachować oryginalne dane do późniejszej analizy porównawczej.

⚠ Warning

Metoda `index()` zgłasza błąd `ValueError` jeśli element nie istnieje. Zawsze sprawdzaj obecność elementu operatorem `in` przed użyciem `index()`.

Wizualizacja danych z list

```
import matplotlib.pyplot as plt

# Dane do wizualizacji: wyniki accuracy z 10 eksperymentów
experiment_numbers = list(range(1, 11))
accuracy_values = [0.72, 0.75, 0.78, 0.82, 0.85, 0.83, 0.87, 0.89, 0.88,
0.91]

# Wykres liniowy - trend uczenia
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.plot(experiment_numbers, accuracy_values, marker='o', linewidth=2,
markersize=8)
plt.xlabel('Numer eksperymentu')
plt.ylabel('Accuracy')
```

```
plt.title('Postęp uczenia modelu')
plt.grid(True, alpha=0.3)
plt.ylim(0.7, 0.95)

# Wykres słupkowy - porównanie min, mean, max
plt.subplot(1, 2, 2)
stats = ['Min', 'Mean', 'Max']
values = [min(accuracy_values),
          sum(accuracy_values)/len(accuracy_values),
          max(accuracy_values)]
plt.bar(stats, values, color=['red', 'blue', 'green'], alpha=0.7)
plt.ylabel('Accuracy')
plt.title('Statystyki wyników')
plt.ylim(0.7, 0.95)

plt.tight_layout()
# plt.savefig('list_visualization.png', dpi=150)  # Zapis do pliku
# plt.show()  # Wyświetlenie wykresu
print("Wykres wygenerowany (kod gotowy do uruchomienia w Colab/Jupyter)")
```

---

3. Łańcuchy Znaków (Stringi) - Operacje i Metody

Definition

String (łańcuch znaków) to **niemutowalna** sekwencja znaków Unicode. W Data Science stringi są kluczowe do przetwarzania nazw kolumn, kategorii, tekstów, ścieżek plików i logów.

Wprowadzenie

Stringi w Pythonie zachowują się podobnie do list (indeksowanie, slicing), ale są **niemutowalne** - nie można zmienić pojedynczego znaku. Wszystkie metody stringów zwracają nowe stringi. W analizie danych stringi są używane do:

- Czyszczenia nazw kolumn (usuwanie spacji, zmiana wielkości liter)
- Parsowania logów i plików tekstowych
- Kategoryzacji danych tekstowych
- Tworzenia raportów i komunikatów

Przykład 1: Tworzenie stringów i podstawowe operacje

```python
# Różne sposoby tworzenia stringów
column_name_single = 'temperature'
column_name_double = "humidity"
multiline_description = """
Ten dataset zawiera pomiary:
- Temperatura (°C)
- Wilgotność (%)
- Ciśnienie (hPa)
"""

print(f"Single quotes: {column_name_single}")
print(f"Double quotes: {column_name_double}")
print(f"Triple quotes:{multiline_description}")

# Konkatenacja stringów
first_name = "Data"
last_name = "Science"
full_name = first_name + " " + last_name
print(f"Konkatenacja: {full_name}")

# Powielanie stringów
separator = "-" * 50
print(separator)
print("Dataset Report")
print(separator)

# f-strings - najlepsza metoda formatowania w Data Science
model_name = "RandomForest"
accuracy = 0.8756
precision = 0.8432
recall = 0.8901

report = f"""
Model: {model_name}
Accuracy:  {accuracy:.2%}
Precision: {precision:.2%}
Recall:    {recall:.2%}
"""
print(report)

# Zaawansowane formatowanie
for i, metric in enumerate([accuracy, precision, recall], 1):
    print(f"Metric {i}: {metric:6.2%} | Raw: {metric:.4f}")
```

Wyjaśnienie: Używamy pojedynczych `'` lub podwójnych `"` cudzysłowów dla krótkich stringów. Potrójne `"""` dla wieloliniowych tekstów. F-strings (`f"{zmienna}"`) to preferowana metoda formatowania - szybka, czytelna i wydajna.

Przykład 2: Indeksowanie i slicing stringów

```
# Nazwa pliku datasetu
filename = "sales_data_2024_Q1.csv"

# Indeksowanie pojedynczych znaków
print(f"Pierwszy znak: {filename[0]}")     # 's'
print(f"Ostatni znak: {filename[-1]}")     # 'v'
print(f"Rozszerzenie pliku: {filename[-4:]}")  # '.csv'

# Slicing do ekstrakcji informacji
year = filename[11:15]
quarter = filename[16:18]
print(f"Rok: {year}, Kwartał: {quarter}")

# Ścieżka do pliku
filepath = "/data/raw/experiments/model_v3_final.pkl"

# Ekstrakcja nazwy pliku z pełnej ścieżki (prosty sposób)
filename_from_path = filepath.split('/')[-1]
print(f"Nazwa pliku: {filename_from_path}")

# Ekstrakcja katalogu
directory = '/'.join(filepath.split('/')[:-1])
print(f"Katalog: {directory}")

# Niemutowalność - nie można zmienić pojedynczego znaku
text = "Data Science"
# text[0] = 'd'  # To spowoduje TypeError!

# Zamiast tego tworzymy nowy string
text_lower = text[0].lower() + text[1:]
print(f"Zmieniony pierwszy znak: {text_lower}")
```

Wyjaśnienie: Stringi obsługują indeksowanie i slicing jak listy, ale są niemutowalne. Każda "zmiana" tworzy nowy string. W Data Science często używamy slicingu do parsowania nazw plików i ścieżek.

Przykład 3: Metody stringów - czyszczenie i transformacja

```python
# Realistyczne nazwy kolumn z datasetu (często mają problemy)
raw_columns = [
    "  Customer_ID  ",
    "TOTAL_AMOUNT",
    "purchase_date",
    "Product Category",
    "  email_address  "
]

print("Oryginalne nazwy kolumn:")
for col in raw_columns:
    print(f"'{col}'")

# Czyszczenie nazw kolumn
cleaned_columns = []
for col in raw_columns:
    # 1. Usunięcie białych znaków z początku i końca
    col = col.strip()
    # 2. Zmiana na małe litery
    col = col.lower()
    # 3. Zamiana spacji na podkreślniki
    col = col.replace(' ', '_')
    cleaned_columns.append(col)

print("\nWyczyszczone nazwy kolumn:")
for col in cleaned_columns:
    print(f"'{col}'")

# Metody transformacji
sample_text = "  Machine Learning Model  "

print(f"strip():  '{sample_text.strip()}'")         # Usuwa białe znaki
print(f"upper():  '{sample_text.upper()}'")         # Wielkie litery
print(f"lower():  '{sample_text.lower()}'")         # Małe litery
print(f"title():  '{sample_text.title()}'")         # Pierwsza litera
każdego słowa wielka
print(f"replace(): '{sample_text.replace('Model', 'Algorithm')}'")

# Praktyczny przykład - normalizacja kategorii
categories_messy = ["  electronics ", "ELECTRONICS", "Electronics  ",
"electronics"]

# Normalizacja do jednej formy
categories_clean = [cat.strip().lower() for cat in categories_messy]
print(f"\nKategorie po normalizacji: {categories_clean}")
```

```
# Zliczenie unikalnych kategorii
unique_categories = list(set(categories_clean))
print(f"Unikalne kategorie: {unique_categories}")
```

Wyjaśnienie: `strip()` usuwa białe znaki (spacje, tabulatory, nowe linie) z początku i końca - KRYTYCZNE przy czyszczeniu danych. `upper()/lower()` normalizują wielkość liter. `replace()` zamienia fragmenty tekstu.

Przykład 4: Metody sprawdzające i wyszukiwanie

```
# Walidacja danych wejściowych

# Przykładowe dane użytkownika
user_inputs = [
    "user_12345",
    "2024-01-15",
    "  ",
    "0.8756",
    "model_v3",
    ""
]

print("Walidacja danych wejściowych:")
for data in user_inputs:
    # Sprawdzenie czy pusty string lub tylko białe znaki
    if not data or data.isspace():
        print(f"  '{data}' -> PUSTY lub BIAŁE ZNAKI")
        continue

    # Sprawdzenie czy zawiera tylko cyfry
    if data.isdigit():
        print(f"  '{data}' -> tylko CYFRY")
    # Sprawdzenie czy zawiera tylko litery
    elif data.isalpha():
        print(f"  '{data}' -> tylko LITERY")
    # Sprawdzenie czy alfanumeryczne
    elif data.replace('_', '').replace('-', '').replace('.', '').isalnum():
        print(f"  '{data}' -> poprawny FORMAT")
    else:
        print(f"  '{data}' -> NIEPRAWIDŁOWY format")

# Metody wyszukiwania
log_entry = "2024-01-15 14:23:45 ERROR Model training failed:
OutOfMemoryError"
```

```python
# Sprawdzenie co zawiera log
if log_entry.startswith("2024"):
    print("\nLog z 2024 roku")

if log_entry.endswith("Error"):
    print("Log kończy się na 'Error'")

if "ERROR" in log_entry:
    print("Log zawiera ERROR")

# Znalezienie pozycji
error_position = log_entry.find("ERROR")
print(f"Słowo 'ERROR' zaczyna się na pozycji: {error_position}")

# Ekstrakcja poziomu logu
if "ERROR" in log_entry:
    level = "ERROR"
elif "WARNING" in log_entry:
    level = "WARNING"
else:
    level = "INFO"

print(f"Poziom logu: {level}")

# count() - zliczanie wystąpień
path = "/data/projects/ml_project/models/v1/model.pkl"
slash_count = path.count('/')
print(f"\nŚcieżka ma {slash_count} separatorów '/'")

# Podział ścieżki na komponenty
path_components = path.split('/')
print(f"Komponenty ścieżki: {path_components}")
```

Wyjaśnienie: Metody `startswith()`, `endswith()`, `in` służą do sprawdzania zawartości. `find()` zwraca pozycję (-1 jeśli nie znaleziono). Metody `isdigit()`, `isalpha()`, `isalnum()` walidują format danych.

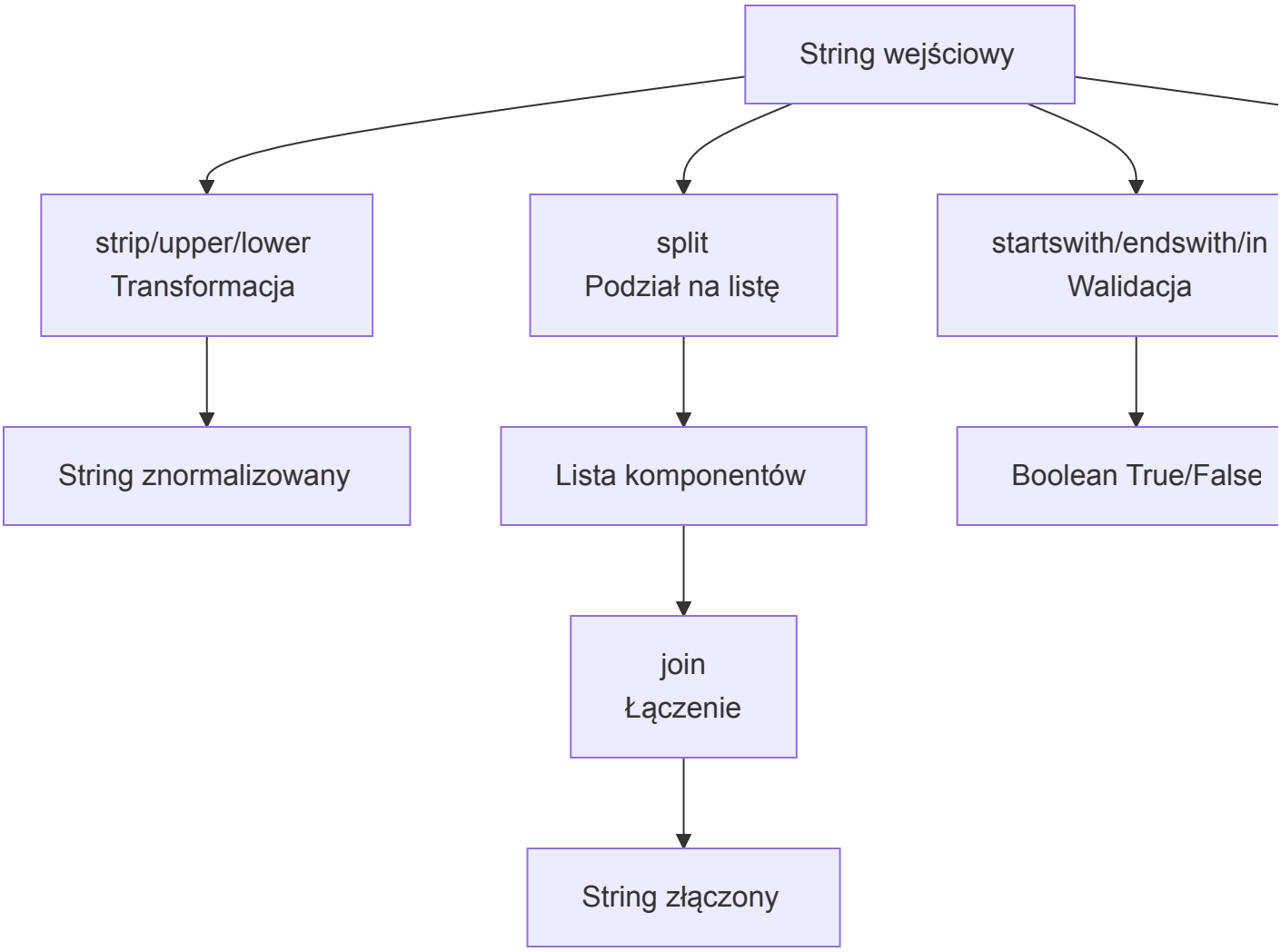
Przykład 5: Split i Join - parsowanie i budowanie tekstów

```python
# Parsowanie CSV (uproszczona wersja bez biblioteki pandas)
csv_line = "John Doe,35,Data Scientist,85000,New York"

# Split - podział na komponenty
fields = csv_line.split(',')
print(f"Pola CSV: {fields}")
```

```python
name, age, job, salary, city = fields
print(f"Nazwa: {name}")
print(f"Wiek: {age}")
print(f"Zawód: {job}")
print(f"Pensja: ${salary}")
print(f"Miasto: {city}")

# Parsowanie konfiguracji
config = "learning_rate=0.001;epochs=100;batch_size=32"
params = config.split(';')
print(f"\nParametry: {params}")

config_dict = {}
for param in params:
    key, value = param.split('=')
    config_dict[key] = value

print(f"Konfiguracja: {config_dict}")

# Join - łączenie elementów listy w string
columns = ['customer_id', 'purchase_date', 'amount', 'category']

# Tworzenie nagłówka CSV
csv_header = ','.join(columns)
print(f"\nNagłówek CSV: {csv_header}")

# Tworzenie SQL query
sql_select = f"SELECT {', '.join(columns)} FROM sales"
print(f"SQL: {sql_select}")

# Raport tekstowy
metrics = ['accuracy', 'precision', 'recall', 'f1_score']
values = [0.85, 0.83, 0.87, 0.85]

report_lines = []
for metric, value in zip(metrics, values):
    report_lines.append(f"{metric}: {value:.2%}")

report = '\n'.join(report_lines)
print(f"\nRaport modelu:\n{report}")

# Budowanie ścieżki (zamiast os.path.join dla prostych przypadków)
path_parts = ['data', 'processed', 'sales_2024.csv']
full_path = '/'.join(path_parts)
print(f"\nŚcieżka: {full_path}")
```

Wyjaśnienie: `split(separator)` dzieli string na listę po separatorze. `join(lista)` łączy elementy listy w string, wstawiając separator między nimi. To fundamentalne operacje przy parsowaniu i budowaniu tekstów w Data Science.

Wizualizacja: Operacje na stringach



Tip

Best practice: Zawsze normalizuj dane tekstowe przed analizą - używaj `strip()`, `lower()` do ujednolicenia formatu. To zapobiega problemom z "Electronics" vs " electronics ".

Warning

Metoda `split()` bez argumentów dzieli po dowolnych białych znakach (spacje, tabulatory, nowe linie) i ignoruje puste fragmenty. `split(' ')` dzieli tylko po pojedynczej spacji i zachowuje puste fragmenty.

Przykład 6: Formatowanie stringów - f-strings zaawansowane

```
# Raport z eksperymentów ML
experiments_data = [
    {'model': 'LogisticRegression', 'accuracy': 0.7823, 'time': 1.234},
    {'model': 'RandomForest', 'accuracy': 0.8956, 'time': 45.678},
    {'model': 'XGBoost', 'accuracy': 0.9234, 'time': 123.456},
    {'model': 'NeuralNetwork', 'accuracy': 0.9456, 'time': 567.891},
]

print("=" * 70)
print(f"{'Model':<20} {'Accuracy':>10} {'Time (s)':>12} {'Score':>10}")
print("=" * 70)

for exp in experiments_data:
    # Wyrównanie tekstu: < lewo, > prawo, ^ środek
    # Formatowanie liczb: .2% procenty, .2f float
    model = f"{exp['model']:<20}"
    acc = f"{exp['accuracy']:>10.2%}"
    time = f"{exp['time']:>12.2f}"

    # Score = accuracy / log(time)
    import math
    score = exp['accuracy'] / math.log10(exp['time'] + 1)
    score_str = f"{score:>10.4f}"

    print(f"{model} {acc} {time} {score_str}")

print("=" * 70)

# Formatowanie dużych liczb
dataset_size = 1_234_567
print(f"\nRozmiar datasetu: {dataset_size:,} rekordów")  # 1,234,567
print(f"W formacie naukowym: {dataset_size:.2e}")  # 1.23e+06

# Formatowanie dla finansów
revenue = 123456.789
print(f"Przychód: ${revenue:,.2f}")  # $123,456.79
```

```
# Padding z zerami
for i in range(1, 13):
    print(f"Eksperyment_{i:03d}.log", end="  ")
    if i % 4 == 0:
        print()  # Nowa linia co 4 elementy
```

Wyjaśnienie: F-strings obsługują zaawansowane formatowanie: wyrównanie (`<`, `>`, `^`), szerokość pola, precyzję, separatory tysięcy (`,`), format procentowy (`%`), naukowy (`e`).

🌱 Aspekt środowiskowy

W tej lekcji zwracamy uwagę na efektywne wykorzystanie struktur danych, co bezpośrednio przekłada się na zużycie zasobów obliczeniowych:

Efektywna praca z listami

```
import sys
import time

# Porównanie efektywności: konkatenacja vs extend

# NIEEFEKTYWNE - wielokrotne kopiowanie listy
def inefficient_build(n):
    result = []
    for i in range(n):
        result = result + [i]  # Tworzy NOWĄ listę za każdym razem!
    return result

# EFEKTYWNE - modyfikacja in-place
def efficient_build(n):
    result = []
    for i in range(n):
        result.append(i)  # Dodaje do istniejącej listy
    return result

# Pomiar czasu i zużycia pamięci
n = 10000

start = time.time()
list1 = inefficient_build(n)
time_inefficient = time.time() - start

start = time.time()
```

```python
list2 = efficient_build(n)
time_efficient = time.time() - start

print(f"Nieefektywna metoda (+): {time_inefficient:.4f} s")
print(f"Efektywna metoda (append): {time_efficient:.4f} s")
print(f"Przyspieszenie: {time_inefficient/time_efficient:.1f}x")

# Oszczędność energii ~ oszczędność czasu obliczeniowego
energy_saved = (time_inefficient - time_efficient) / time_inefficient * 100
print(f"Oszczędność energii: ~{energy_saved:.1f}%")
```

Efektywne operacje na stringach

```python
# NIEEFEKTYWNE - konkatenacja w pętli
def build_report_bad(n):
    report = ""
    for i in range(n):
        report += f"Line {i}\n"  # Tworzy nowy string za każdym razem!
    return report

# EFEKTYWNE - join z listą
def build_report_good(n):
    lines = []
    for i in range(n):
        lines.append(f"Line {i}")
    return '\n'.join(lines)

# Pomiar wydajności
n = 5000

start = time.time()
report1 = build_report_bad(n)
time_bad = time.time() - start

start = time.time()
report2 = build_report_good(n)
time_good = time.time() - start

print(f"\nKonkatenacja stringów w pętli: {time_bad:.4f} s")
print(f"Join z listą: {time_good:.4f} s")
print(f"Przyspieszenie: {time_bad/time_good:.1f}x")
```

Kluczowe zasady Green IT dla kolekcji:

1. **Używaj list comprehensions** zamiast pętli for z append (szybsze i bardziej czytelne)

2. **Preferuj `extend()` i `append()`** zamiast operatora `+` dla list
3. **Używaj `join()` zamiast `+=`** przy budowaniu długich stringów
4. **Unikaj niepotrzebnego kopiowania** - modyfikuj in-place gdy to możliwe
5. **Czyść duże struktury** (`del` , `clear()`) gdy nie są już potrzebne
6. **Używaj slicingu** zamiast pętli do filtrowania danych

> ✓ **Success**
>
> Efektywny kod = mniej czasu obliczeń = mniej zużytej energii = mniejszy ślad węglowy. W Data Science, gdzie przetwarzamy miliony rekordów, różnica między `+=` a `join()` może oszczędzić godziny czasu procesora!

📊 Praca z Prawdziwymi Danymi

Dataset: Iris (klasyczny dataset ML)

Pokażemy jak pracować z danymi Iris używając tylko list i stringów (bez Pandas), aby zrozumieć fundamenty przetwarzania danych.

```
# Iris dataset - uproszczona wersja (5 pierwszych rekordów)
# Format: sepal_length, sepal_width, petal_length, petal_width, species

iris_data_raw = """5.1,3.5,1.4,0.2,setosa
4.9,3.0,1.4,0.2,setosa
4.7,3.2,1.3,0.2,setosa
4.6,3.1,1.5,0.2,setosa
5.0,3.6,1.4,0.2,setosa
7.0,3.2,4.7,1.4,versicolor
6.4,3.2,4.5,1.5,versicolor
6.9,3.1,4.9,1.5,versicolor
5.5,2.3,4.0,1.3,versicolor
6.5,2.8,4.6,1.5,versicolor
6.3,3.3,6.0,2.5,virginica
5.8,2.7,5.1,1.9,virginica
7.1,3.0,5.9,2.1,virginica
6.3,2.9,5.6,1.8,virginica
6.5,3.0,5.8,2.2,virginica"""

# Parsowanie danych
lines = iris_data_raw.strip().split('\n')
print(f"Liczba rekordów: {len(lines)}")
```

```python
# Przygotowanie struktur danych
column_names = ['sepal_length', 'sepal_width', 'petal_length',
'petal_width', 'species']
data = []

for line in lines:
    # Split po przecinku
    values = line.split(',')

    # Konwersja numerycznych wartości
    record = {
        'sepal_length': float(values[0]),
        'sepal_width': float(values[1]),
        'petal_length': float(values[2]),
        'petal_width': float(values[3]),
        'species': values[4]
    }
    data.append(record)

print(f"\nPierwszy rekord: {data[0]}")
print(f"Ostatni rekord: {data[-1]}")

# Analiza: obliczenie średnich dla każdej cechy numerycznej
numeric_columns = ['sepal_length', 'sepal_width', 'petal_length',
'petal_width']

print("\nŚrednie wartości cech:")
for col in numeric_columns:
    # Ekstrakcja wartości kolumny
    values = [record[col] for record in data]

    # Obliczenie statystyk
    mean_val = sum(values) / len(values)
    min_val = min(values)
    max_val = max(values)

    print(f"  {col:15s}: mean={mean_val:.2f}, min={min_val:.2f}, max=
{max_val:.2f}")

# Analiza rozkładu klas
species_list = [record['species'] for record in data]
unique_species = []
for sp in species_list:
    if sp not in unique_species:
        unique_species.append(sp)
```

```python
print(f"\nUnikalne gatunki: {unique_species}")

for sp in unique_species:
    count = species_list.count(sp)
    percentage = (count / len(species_list)) * 100
    print(f"  {sp}: {count} rekordów ({percentage:.1f}%)")

# Filtrowanie danych - tylko 'setosa'
setosa_data = [record for record in data if record['species'] == 'setosa']
print(f"\nLiczba rekordów 'setosa': {len(setosa_data)}")

# Średnia długość płatka dla setosa
setosa_petal_lengths = [record['petal_length'] for record in setosa_data]
mean_petal_setosa = sum(setosa_petal_lengths) / len(setosa_petal_lengths)
print(f"Średnia długość płatka (setosa): {mean_petal_setosa:.2f} cm")
```

Wizualizacja Iris dataset

```python
import matplotlib.pyplot as plt

# Przygotowanie danych do wizualizacji
species_names = unique_species
sepal_lengths_by_species = {}
petal_lengths_by_species = {}

for sp in species_names:
    # Filtruj dane dla każdego gatunku
    sp_data = [rec for rec in data if rec['species'] == sp]

    # Ekstrakcja cech
    sepal_lengths_by_species[sp] = [rec['sepal_length'] for rec in sp_data]
    petal_lengths_by_species[sp] = [rec['petal_length'] for rec in sp_data]

# Wykres
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Wykres 1: Box plot dla długości działek kielicha
positions = [1, 2, 3]
ax1.boxplot([sepal_lengths_by_species[sp] for sp in species_names],
            labels=species_names, positions=positions)
ax1.set_xlabel('Gatunek')
ax1.set_ylabel('Długość działki kielicha (cm)')
ax1.set_title('Rozkład długości działek kielicha wg gatunku')
ax1.grid(True, alpha=0.3)
```

```python
# Wykres 2: Scatter plot - sepal vs petal length
colors = {'setosa': 'red', 'versicolor': 'blue', 'virginica': 'green'}
for sp in species_names:
    sp_data = [rec for rec in data if rec['species'] == sp]
    x = [rec['sepal_length'] for rec in sp_data]
    y = [rec['petal_length'] for rec in sp_data]
    ax2.scatter(x, y, c=colors[sp], label=sp, alpha=0.6, s=100)

ax2.set_xlabel('Długość działki kielicha (cm)')
ax2.set_ylabel('Długość płatka (cm)')
ax2.set_title('Zależność: długość działki kielicha vs długość płatka')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
# plt.savefig('iris_analysis.png', dpi=150)
# plt.show()
print("\nWykres gotowy (kod do uruchomienia w Jupyter/Colab)")
```

🧪 Zadania do samodzielnej pracy

✏️ Zadania podstawowe (1-8)

1. ✏️ **Zadanie 1 – Tworzenie i indeksowanie listy**

   Stwórz listę zawierającą pomiary temperatury z 7 dni: `[22.5, 23.0, 21.5, 24.0, 23.5, 22.0, 23.0]`. Wyświetl:

   - Temperaturę z pierwszego dnia
   - Temperaturę z ostatniego dnia
   - Temperatury z trzech środkowych dni (slicing)

   (proste)

2. ✏️ **Zadanie 2 – Dodawanie elementów do listy**

   Zacznij od pustej listy `results = []`. Dodaj do niej 5 wartości accuracy: 0.75, 0.82, 0.79, 0.85, 0.88. Następnie:

   - Wyświetl długość listy
   - Wyświetl średnią wartość
   - Wyświetl maksymalną wartość

(proste)

3. ✏️ **Zadanie 3 – Sortowanie listy**

   Masz listę wyników: `[0.78, 0.92, 0.85, 0.88, 0.81]`. Posortuj ją malejąco i wyświetl 3 najlepsze wyniki.

   (proste)

4. ✏️ **Zadanie 4 – Czyszczenie nazw kolumn**

   Masz listę nazw kolumn: `[" Customer ID ", "TOTAL AMOUNT", " date "]`. Wyczyść je zgodnie z konwencją:

   - Usuń białe znaki (strip)
   - Zamień na małe litery (lower)
   - Zamień spacje na podkreślniki (replace)

   (proste)

5. ✏️ **Zadanie 5 – Parsowanie CSV**

   Masz string CSV: `"Alice,28,Data Scientist,75000"`. Użyj `split(',')` aby wyciągnąć:

   - Imię
   - Wiek (jako int)
   - Zawód
   - Pensję (jako int)

   Wyświetl te informacje w formacie: `"Alice ma 28 lat, pracuje jako Data Scientist i zarabia $75000"`

   (proste)

6. ✏️ **Zadanie 6 – Formatowanie raportu**

   Masz dane: `model_name = "XGBoost"`, `accuracy = 0.8934`, `time = 45.67`. Użyj f-string aby stworzyć raport:

   ```
   Model: XGBoost
   Accuracy: 89.34%
   Training time: 45.67s
   ```

   (proste)

7. ✏️ **Zadanie 7 – Zliczanie wystąpień**

   Masz listę predykcji: `['spam', 'ham', 'spam', 'spam', 'ham', 'ham', 'spam']`. Policz ile jest 'spam' i ile 'ham' używając metody `count()`.

(proste)

8. ✏️ **Zadanie 8 – Ekstakcja informacji ze stringa**

   Masz nazwę pliku: `"model_results_2024_Q3.csv"`. Wyciągnij rok i kwartał używając slicing lub split.

   (proste)

✏️ Zadania średnie (9-12)

9. ✏️ **Zadanie 9 – Filtrowanie outlierów**

   Masz listę pomiarów: `[23, 24, 22, 150, 23, 25, -10, 24, 23]`. Outlierem jest wartość poza zakresem [20, 30]. Stwórz nową listę bez outlierów używając list comprehension.

   (średnie)

10. ✏️ **Zadanie 10 – Agregacja danych po kategoriach**

    Masz listę transakcji (kategoria, kwota):

```
transactions = [
    ('food', 50),
    ('transport', 20),
    ('food', 30),
    ('entertainment', 40),
    ('transport', 15),
    ('food', 25)
]
```

    Oblicz łączną kwotę dla każdej kategorii. Wynik zapisz w słowniku (użyj pętli i warunków).

    (średnie)

11. ✏️ **Zadanie 11 – Normalizacja tekstów**

    Masz listę kategorii produktów z różnymi formatami:

```
categories = ["  Electronics  ", "ELECTRONICS", "electronics", "  Books", "books  ", "BOOKS"]
```

    Znormalizuj je (strip + lower) i znajdź unikalne kategorie.

    (średnie)

12. ✏️ **Zadanie 12 – Tworzenie SQL query**

    Masz:

- Tabelę: `"customers"`
- Kolumny do wybrania: `['id', 'name', 'email', 'city']`
- Warunek: `"city = 'New York'"`

Zbuduj string z query SQL używając `join()`:

```
SELECT id, name, email, city FROM customers WHERE city = 'New York'
```

(średnie)

🧠 Zadania wyzwanie (13-20)

13. 🧠 **Zadanie 13 – Analiza Iris dataset**

    Użyj danych Iris z sekcji "Praca z Prawdziwymi Danymi". Dla każdego gatunku oblicz:

    - Średnią długość płatka (petal_length)
    - Maksymalną szerokość płatka (petal_width)
    - Stosunek długości płatka do szerokości

    Wyświetl wyniki w czytelnym formacie tabelarycznym używając f-strings z wyrównaniem.

    (challenge)

14. 🧠 **Zadanie 14 – Ranking modeli**

    Masz listę wyników modeli:

```
models = [
    ('LogisticRegression', 0.78, 1.2),
    ('RandomForest', 0.92, 45.6),
    ('SVM', 0.85, 12.3),
    ('XGBoost', 0.95, 67.8)
]
# Format: (nazwa, accuracy, czas_treningu_s)
```

    Stwórz ranking modeli według "efficiency score" = accuracy / log10(czas). Posortuj malejąco i wyświetl top 3.

    (challenge)

15. 🧠 **Zadanie 15 – Parsowanie logów**

    Masz listę logów:

```
logs = [
    "2024-01-15 10:23:45 INFO Model training started",
    "2024-01-15 10:45:12 WARNING Low memory",
    "2024-01-15 11:02:33 ERROR Training failed",
    "2024-01-15 11:05:01 INFO Retrying with smaller batch"
]
```

Wyciągnij i zlicz poziomy logów (INFO, WARNING, ERROR). Oblicz % każdego poziomu.

(challenge)

16. 🧠 **Zadanie 16 – Confusion Matrix z list**

Masz listę prawdziwych etykiet i predykcji:

```
y_true = [1, 0, 1, 1, 0, 1, 0, 0, 1, 0]
y_pred = [1, 0, 1, 0, 0, 1, 0, 1, 1, 0]
```

Oblicz TP, TN, FP, FN używając pętli i warunków. Następnie oblicz accuracy, precision, recall.

(challenge)

17. 🧠 **Zadanie 17 – Czyszczenie danych CSV**

Masz surowe linie CSV z błędami:

```
csv_lines = [
    "id,name,age,city",
    "1,Alice,28,  New York  ",
    "2,  Bob  ,, Boston",
    "3,Charlie,35,",
    "4,Diana,29,Chicago"
]
```

Wyczyść dane:

- Usuń białe znaki z wartości
- Zamień puste wartości na "Unknown"
- Zbuduj listę słowników (każda linia = słownik)

(challenge)

18. 🧠 **Zadanie 18 – Agregacja z grupowaniem**

Masz dane sprzedaży (produkt, kategoria, kwota):

```
sales = [
    ('Laptop', 'Electronics', 1200),
    ('Book', 'Media', 15),
    ('Phone', 'Electronics', 800),
    ('DVD', 'Media', 10),
    ('Tablet', 'Electronics', 400),
    ('Magazine', 'Media', 5)
]
```

Oblicz:

- Łączną sprzedaż dla każdej kategorii
- Średnią cenę produktu w każdej kategorii
- Liczbę produktów w każdej kategorii

Wynik jako słownik słowników.

(challenge)

19. 🧠 **Zadanie 19 – Generowanie raportu markdown**

Masz dane eksperymentów:

```
experiments = [
    {'name': 'Exp_01', 'accuracy': 0.85, 'precision': 0.83, 'recall':
0.87},
    {'name': 'Exp_02', 'accuracy': 0.89, 'precision': 0.88, 'recall':
0.90},
    {'name': 'Exp_03', 'accuracy': 0.87, 'precision': 0.85, 'recall':
0.89}
]
```

Wygeneruj raport w formacie markdown z tabelą:

```
| Experiment | Accuracy | Precision | Recall |
|------------|----------|-----------|--------|
| Exp_01     | 85.00%   | 83.00%    | 87.00% |
...
```

Użyj list comprehension i join.

(challenge)

20. 🧠 **Zadanie 20 – Walidacja i transformacja pipeline**

Stwórz funkcję `clean_and_validate(data)`, która dla listy stringów:

```
raw_data = [
    "  0.8567  ",
    "invalid",
    "0.9123",
    "",
    "  0.7890  ",
    "0.95",
    "not_a_number"
]
```

Wykonuje:

1. Strip białych znaków
2. Walidację czy string reprezentuje liczbę (użyj try/except)
3. Konwersję na float
4. Filtrowanie wartości poza zakresem [0, 1]
5. Zwraca listę poprawnych wartości float oraz listę błędów (indeks, wartość, powód)

(challenge)

📚 Podsumowanie

W tej lekcji nauczyłeś się:

Kluczowe koncepcje:

1. **Listy**:
   - Tworzenie, indeksowanie (dodatnie i ujemne), slicing
   - Mutowalność - modyfikacja elementów in-place
   - Metody: `append()`, `extend()`, `insert()`, `remove()`, `pop()`, `sort()`, `sorted()`, `reverse()`
   - Przeszukiwanie: `index()`, `count()`, `in`
2. **Stringi**:
   - Tworzenie (pojedyncze, podwójne, potrójne cudzysłowy)
   - Niemutowalność - każda operacja tworzy nowy string
   - Transformacje: `strip()`, `upper()`, `lower()`, `title()`, `replace()`
   - Walidacja: `startswith()`, `endswith()`, `in`, `isdigit()`, `isalpha()`
   - Parsowanie: `split()`, `join()`
   - Formatowanie: f-strings z zaawansowanym formatowaniem

3. **Różnice między listami a stringami**:
   - Listy: mutowalne, mogą zawierać różne typy, dynamiczne
   - Stringi: niemutowalne, tylko znaki, używane do tekstu i kategorii
4. **Zastosowania w Data Science**:
   - Przechowywanie i przetwarzanie danych przed użyciem NumPy/Pandas
   - Czyszczenie nazw kolumn i kategorii
   - Parsowanie plików CSV, logów, konfiguracji
   - Budowanie zapytań SQL i raportów
   - Walidacja i transformacja danych tekstowych

Aspekt środowiskowy:

- Efektywne metody (`append`, `extend`, `join`) oszczędzają czas i energię
- Unikanie kopiowania (operator `+` dla list) redukuje zużycie pamięci
- Czytelny kod = mniej debugowania = mniej zużytej energii

Przygotowanie do następnych lekcji:

W następnej lekcji poznasz **słowniki, zbiory i krotki** - bardziej zaawansowane struktury danych, które są kluczowe w analizie i przetwarzaniu danych. Zrozumienie list i stringów jest fundamentem do efektywnej pracy z tymi strukturami.

📖 Dodatkowe Zasoby

Dokumentacja Python:

- Listy (Lists) - oficjalna dokumentacja
- Stringi (Strings) - wszystkie metody stringów
- Format Specification Mini-Language - zaawansowane formatowanie

Tutoriale:

- Real Python - Lists and Tuples - kompleksowy przewodnik
- Real Python - String Formatting - wszystko o f-strings
- Python String Methods - przegląd wszystkich metod

Praktyka:

- LeetCode - Easy Array Problems - zadania z listami
- HackerRank - Python Strings - praktyka ze stringami

- Kaggle - Titanic Dataset - do praktyki parsowania i czyszczenia danych

Best Practices:

- PEP 8 - Style Guide - konwencje nazewnictwa i formatowania
- Python Performance Tips - optymalizacja kodu
- Green Algorithms - kalkulator śladu węglowego dla kodu

Polecane datasety do praktyki:

- **Iris** - klasyfikacja (użyty w tej lekcji)
- **Titanic** - klasyfikacja binarna z wieloma cechami tekstowymi
- **Wine Quality** - regresja/klasyfikacja z czyszczeniem danych
- **IMDB Reviews** - przetwarzanie tekstów